Every Mathematical Equation in the Thesis — Explained

A Symbol-by-Symbol Breakdown

From Spatiotemporal Tokenization to HAF-STD Net

This document takes **every equation** from the thesis methodology and explains it in plain language with concrete numbers.

25 equations across 6 topic areas: Tokenization · V-JEPA 2 Training · Qwen2.5-VL · PCA · HAF-STD Net · Training Techniques

September 30, 2026

Contents

1 V-JEPA 2: Turning Video into Tokens **2**
1.1 Equation 1 — Spatiotemporal Tokenization (Eq. 3.1) 2

2 V-JEPA 2: How It Learns (Training Objectives) **4**
2.1 Equation 2 — Exponential Moving Average / EMA (Eq. 3.2) 4
2.2 Equation 3 — Vision Transformer Block (Eq. 3.3) 5
2.3 Equation 4 — V-JEPA 2 Training Objective (Eq. 3.4) 6
2.4 Equation 5 — Dense Per-Token Prediction Loss (Eq. 3.5) 7
2.5 Equation 6 — Context Loss (Eq. 3.6) 7
2.6 Equation 7 — Total V-JEPA 2.1 Loss (Eq. 3.7) 8
2.7 Equation 8 — Multi-Layer Feature Extraction (Eq. 3.8) 8

3 Qwen2.5-VL: The Vision-Language Model **10**
3.1 Equation 9 — Patch Embedding (Eq. 3.9) 10
3.2 Equation 10 — Window Attention (Eq. 3.10) 10
3.3 Equation 11 — Multimodal Rotary Position Embedding / MRoPE (Eq. 3.11) 11
3.4 Equation 12 — Visual-Language Fusion (Eq. 3.12) 12
3.5 Equation 13 — Language Model Training Objective (Eq. 3.13) 13
3.6 Equation 14 — NF4 Quantization (Eq. 3.14) 14
3.7 Equation 15 — Qwen Feature Extraction (Eq. 3.15) 14

4 Feature Preprocessing: Frozen Protocol and PCA **16**
4.1 Equation 16 — Frozen Feature Protocol 16
4.2 Equation 17 — Standardization (Eq. 3.16) 17
4.3 Equations 18–20 — PCA (Eqs. 3.17–3.19) 17

5 HAF-STD Net: The Novel Classifier **20**
5.1 Equation 21 — Haar Wavelet Transform 20
5.2 Equation 22 — Depth Representation Velocity 21
5.3 Equation 23 — Adaptive Routing Fusion 21
5.4 Equation 24 — Cosine Similarity Classification Head 22

6 Training Techniques **24**
6.1 Equation 25 — Label Smoothing 24
6.2 Equation 26 — Feature-Space Mixup 24
6.3 Equation 27 — 4D Mixup for HAF-STD Net 25

7 Complete Equation Index **26**

V-JEPA 2: Turning Video into Tokens

Equation 1 — Spatiotemporal Tokenization (Eq. 3.1)

The Equation

$$\mathbf{z}_0 = \text{Conv3D}(\mathbf{V}) + \mathbf{E}_{\text{pos}}, \quad \mathbf{z}_0 \in \mathbb{R}^{N \times D}$$

What Every Symbol Means

$\mathbf{V}$	The raw input video. It has shape $\mathbb{R}^{T \times H \times W \times 3}$, meaning T frames, each of height H, width W, with 3 color channels (RGB). In your thesis: $T = 8$, $H = W = 224$, so $\mathbf{V}$ is an $8 \times 224 \times 224 \times 3$ tensor.
$\text{Conv3D}(\cdot)$	A 3D convolution operation. It slides a small 3D filter (size $2 \times 16 \times 16$) over the video volume. This filter groups 2 consecutive frames and a 16×16 spatial patch into one "tubelet." Each tubelet is converted into a single vector.
$\mathbf{E}_{\text{pos}}$	Positional embeddings — extra information added to tell the model *where* each token comes from (its position in time and space). Uses 3D Rotary Position Embedding (RoPE).
$\mathbf{z}_0$	The output: a sequence of token vectors. The entire video is now represented as N vectors, each of dimension D.
N	Total number of tokens $= \frac{T}{2} \times \frac{H}{16} \times \frac{W}{16}$.
D	Dimension of each token vector. For ViT-Huge: $D = 1280$.

Computing N with Your Actual Numbers

$$N = \frac{8}{2} \times \frac{224}{16} \times \frac{224}{16} = 4 \times 14 \times 14 = \mathbf{784} \text{ tokens}$$

So a single 8-frame video clip becomes a sequence of **784 vectors**, each of dimension **1280**.
The shape is: $\mathbf{z}_0 \in \mathbb{R}^{784 \times 1280}$

Plain English Analogy

Think of it like cutting a book into sentences.
The video is like a book with pictures. Conv3D cuts it into small rectangular "chunks" (tubelets). Each chunk covers a small spatial area (16×16 pixels) over a short time window (2 frames). Each chunk is then summarized as a single 1280-number description vector.
The positional embedding $\mathbf{E}_{\text{pos}}$ is like page numbers and paragraph numbers — it tells the model "this chunk came from the top-left of frame 3" so it knows the

spatial and temporal arrangement.

V-JEPA 2: How It Learns (Training Objectives)

Equation 2 — Exponential Moving Average / EMA (Eq. 3.2)

The Equation

$$\bar{\theta} \leftarrow \tau\bar{\theta} + (1-\tau)\theta, \quad \tau \in [0.996, 1.0)$$

What Every Symbol Means

θ	The weights of the **context encoder** (x-Encoder). This encoder is trained normally with gradient descent — it learns from mistakes.
$\bar{\theta}$	The weights of the **target encoder** (y-Encoder). This encoder is *not* trained by gradients. Instead, its weights slowly follow θ.
τ	The "momentum" parameter (between 0.996 and 1.0). It controls how slowly $\bar{\theta}$ follows θ.
$\leftarrow$	"Update to" — the left side gets a new value computed from the right side.

Numerical Example with $\tau = 0.999$

Suppose one particular weight value is:

- Context encoder weight: $\theta = 0.50$
- Target encoder weight (before update): $\bar{\theta}_{\text{old}} = 0.42$

After the EMA update:

$$\bar{\theta}_{\text{new}} = 0.999 \times 0.42 + 0.001 \times 0.50 = 0.41958 + 0.0005 = 0.4201$$

The target encoder barely changed! It moved from 0.42 to 0.4201 — just 0.02% toward θ. This extreme slowness creates a **stable prediction target** that doesn't jump around during training.

Why Do This?

Imagine a teacher and a student.
The student (context encoder θ) learns actively from exercises. The teacher (target encoder $\bar{\theta}$) slowly absorbs what the student learns. The teacher provides stable

"answer keys" for the student to learn from. If the teacher changed as fast as the student, the "answer keys" would be unreliable and the student would be chasing a moving target.
Without EMA, both encoders could "collapse" — they'd learn to output the same constant vector for every input (useless).

Equation 3 — Vision Transformer Block (Eq. 3.3)

The Equation

$$\mathbf{z}'_l = \text{MHSA}(\text{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1}, \qquad \mathbf{z}_l = \text{FFN}(\text{LN}(\mathbf{z}'_l)) + \mathbf{z}'_l$$

What Every Symbol Means

$\mathbf{z}_{l-1}$	Input to layer l (output of the previous layer). A sequence of N token vectors.
$\text{LN}(\cdot)$	Layer Normalization — scales each token to have mean 0 and variance 1. Stabilizes training.
$\text{MHSA}(\cdot)$	Multi-Head Self-Attention — each token "looks at" every other token to figure out which ones are relevant. This is how the model learns relationships between different parts of the video.
$\mathbf{z}'_l$	Intermediate result after attention + residual connection.
$+\mathbf{z}_{l-1}$	**Residual (skip) connection** — adds the original input back. This prevents the "vanishing gradient" problem in deep networks.
$\text{FFN}(\cdot)$	Feed-Forward Network — two linear layers with a nonlinearity (GELU) in between. Transforms each token independently.
$\mathbf{z}_l$	Final output of layer l.

What Happens in One Transformer Layer

Two stages happen in sequence:
Stage 1 (Attention): Every token asks: "Which other tokens should I pay attention to?" A token representing the gymnast's hands might attend to tokens representing the balance beam. The output is a new version of each token that incorporates information from related tokens.
Stage 2 (FFN): Each token is individually transformed through a small neural network. This is where the model applies "thinking" — combining the gathered information into a more useful representation.
The + signs (residual connections) ensure information from earlier layers can flow through without being lost. V-JEPA 2 has 32 such layers ($L = 32$ for ViT-Huge), so the video tokens get progressively refined through 32 rounds of attention + FFN.

Equation 4 — V-JEPA 2 Training Objective (Eq. 3.4)

The Equation

$$\underset{\theta,\,\phi,\,\Delta_y}{\text{minimize}} \quad \|P_\phi(\Delta_y, E_\theta(x)) - \text{sg}(E_{\bar{\theta}}(y))\|_1$$

What Every Symbol Means

x, y	Two different "views" (augmented versions) of the same video clip.
$E_\theta(x)$	Context encoder processes the full unmasked view x. Produces representations for all visible tokens.
$E_{\bar{\theta}}(y)$	Target encoder processes the complete view y. Produces "ground truth" target representations.
$\text{sg}(\cdot)$	**Stop-gradient operator**. This is crucial — it means "use this value but do NOT send gradients back through it." The target encoder is only updated by EMA (Eq. 3.2), never by gradient descent.
Δ_y	Learnable mask tokens that tell the predictor which spatial/temporal positions were masked out.
$P_\phi(\cdot)$	Predictor network — a smaller transformer that takes the context encoder's output and the mask positions, and tries to predict what the target encoder would output for the masked regions.
$\|\cdot\|_1$	L_1 norm (absolute value of differences, summed). Measures how "far apart" the prediction is from the target.
θ, ϕ	Trainable parameters of the context encoder and predictor, respectively.

The Core Idea of V-JEPA 2

"Fill in the blanks" for video.
Imagine you cover up 75% of a video (random spatiotemporal patches). The model must **predict what the hidden parts look like** — not at the pixel level, but in a learned abstract representation space. It's like asking: "Given the gymnast's position in frames 1–4, what would frames 5–8 look like in concept?"
The key insight: by learning to predict in **latent space** (not pixel space), the model learns *semantic* features ("gymnast is flipping") rather than low-level texture.

Equation 5 — Dense Per-Token Prediction Loss (Eq. 3.5)

The Equation

$$\mathcal{L}_{\text{predict}} = \frac{1}{|M|} \sum_{i \in M} \| P_\phi(E_\theta(x), \Delta_y)_i - \text{sg}(E_{\bar{\theta}}(y)_i) \|_1$$

What This Adds Over Eq. 3.4

M	The set of all masked token indices (positions that were hidden).
$\|M\|$	Size of that set — how many tokens were masked.
$\sum_{i \in M}$	Sum over each masked position individually.
$(\cdot)_i$	The subscript i means "the representation at token position i."
$\frac{1}{\|M\|}$	Average over all masked positions — so the loss doesn't depend on how many tokens were masked.

The key difference from Eq. 3.4: V-JEPA 2.1 computes the loss *separately for each masked token*, then averages. This gives the model a **dense, per-position learning signal** — every masked patch gets its own gradient, forcing fine-grained spatial understanding.

Equation 6 — Context Loss (Eq. 3.6)

The Equation

$$\mathcal{L}_{\text{context}} = \frac{1}{|C|} \sum_{j \in C} \lambda_d(j) \cdot \| E_\theta(x)_j - \text{sg}(E_{\bar{\theta}}(y)_j) \|_1$$

What Every Symbol Means

C	The set of **visible** (unmasked) context token indices.
$\lambda_d(j)$	A distance-dependent weight. Visible tokens that are *close* to masked regions get **higher weight**. Tokens far from any mask get lower weight.
Purpose	This forces visible tokens near masked boundaries to have accurate representations too — not just the masked ones.

Equation 7 — Total V-JEPA 2.1 Loss (Eq. 3.7)

The Equation

$$\mathcal{L}_{\text{V-JEPA 2.1}} = \mathcal{L}_{\text{predict}} + \lambda_d \cdot \mathcal{L}_{\text{context}}$$

Plain English

The total training loss is the sum of two parts:

1. $\mathcal{L}_{\text{predict}}$: How well can we predict the masked tokens? (primary task)
2. $\lambda_d \cdot \mathcal{L}_{\text{context}}$: Are the visible tokens near masked regions also good? (auxiliary regularizer)

λ_d is a scalar weight that controls how much importance the context loss gets relative to the prediction loss.

Equation 8 — Multi-Layer Feature Extraction (Eq. 3.8)

The Equation

$$\mathbf{F}_{\text{multi}} = [\bar{\mathbf{z}}_{L/4};\ \bar{\mathbf{z}}_{L/2};\ \bar{\mathbf{z}}_{3L/4};\ \bar{\mathbf{z}}_{L}] \in \mathbb{R}^{4\times T'\times D}$$

What Every Symbol Means

L	Total number of transformer layers (32 for ViT-Huge).
$L/4, L/2, 3L/4, L$	Layers 8, 16, 24, 32 — features from 25%, 50%, 75%, 100% depth.
$\bar{\mathbf{z}}_l$	Features from layer l after **spatial mean pooling** (averaging over the $14 \times 14 = 196$ spatial positions within each temporal frame). Shape: $\mathbb{R}^{T'\times D}$.
T'	Number of temporal tokens after tubelet grouping. $T' = T/2 = 8/2 = 4$ temporal positions.
$[\cdot;\cdot;\cdot;\cdot]$	Stacking the four layers together.
$\mathbb{R}^{4\times T'\times D}$	Final shape: $4 \times 4 \times 1280 = 4$ layers $\times$ 4 time steps $\times$ 1280 dimensions.

Concrete Shape

$\mathbf{F}_{\text{multi}} \in \mathbb{R}^{4\times 4\times 1280}$
This is a 3D tensor with 20,480 numbers total. It contains features from 4 different depths of the network, at 4 different time points, each described by 1280 numbers. This is the input to the HAF-STD Net classifier.

Why Multiple Layers?

Early layers (layer 8) capture low-level features: edges, textures, simple motions. Middle layers (16, 24) capture mid-level features: body parts, apparatus shapes. Final layer (32) captures high-level semantic features: "this is a vault" vs "this is a beam routine."
By using **all four**, the HAF-STD Net gets a richer, multi-scale view of the video than any single layer alone.

Qwen2.5-VL: The Vision-Language Model

Equation 9 — Patch Embedding (Eq. 3.9)

The Equation

$$\mathbf{v}_0 = \text{Conv3D}(\mathbf{V}_{\text{frames}}), \quad \mathbf{v}_0 \in \mathbb{R}^{N_v \times D_v}$$

What Every Symbol Means

$\mathbf{V}_{\text{frames}}$	The input video frames (4 frames in this thesis).
$\text{Conv3D}(\cdot)$	3D convolution that converts raw pixels into patch tokens. Uses 14×14 spatial patches and temporal stride of 2.
N_v	Number of visual tokens. Unlike V-JEPA 2 (fixed), this **varies dynamically** based on input resolution.
D_v	Hidden dimension of the vision encoder.

This is conceptually similar to V-JEPA's Eq. 3.1 but with a key difference: Qwen preserves the **original resolution** of each frame instead of resizing everything to 224×224.

Equation 10 — Window Attention (Eq. 3.10)

The Equation

$$\text{WindowAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}_w \mathbf{K}_w^T}{\sqrt{d_k}}\right) \mathbf{V}_w$$

What Every Symbol Means

$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	Query, Key, Value matrices — the three projections that every attention mechanism computes from input tokens. (Note: $\mathbf{V}$ here means "Value," not "video"!)
$\mathbf{Q}_w, \mathbf{K}_w, \mathbf{V}_w$	The subscript w means these are restricted to tokens **within the same spatial window** (e.g., a 112×112 pixel region).
$\mathbf{Q}_w \mathbf{K}_w^T$	Matrix multiplication of queries with keys. This computes "compatibility scores" — how much each token should attend to each other token.
$\sqrt{d_k}$	Scaling factor (d_k = dimension of each key). Prevents the dot products from becoming too large, which would make softmax produce extreme (near-0 or near-1) values.
$\text{softmax}(\cdot)$	Converts raw scores into probabilities that sum to 1. Each token gets a probability distribution over which other tokens to attend to.
$\cdot\, \mathbf{V}_w$	Multiply the attention probabilities by the values. This produces a weighted average of the value vectors — each token's output is a "summary" of the tokens it attended to.

Why "Window" Attention?

Standard (global) self-attention lets every token attend to **every** other token. With N tokens, this requires N^2 computations — quadratic scaling.
With 1000 tokens: $1000^2 = 1{,}000{,}000$ computations.
With 10,000 tokens: $10{,}000^2 = 100{,}000{,}000$ computations. Too expensive!
Window attention restricts each token to only attend within its local $w \times w$ window.
Cost becomes $N \times w^2$ — **linear in** N, not quadratic.

Equation 11 — Multimodal Rotary Position Embedding / MRoPE (Eq. 3.11)

The Equation

$$\text{MRoPE}(\mathbf{x}, t, h, w) = \mathbf{x} \odot [\cos(\theta_t), \cos(\theta_h), \cos(\theta_w)] + \hat{\mathbf{x}} \odot [\sin(\theta_t), \sin(\theta_h), \sin(\theta_w)]$$

What Every Symbol Means

$\mathbf{x}$	The input token vector.
t, h, w	Three position coordinates: time, height, width.
$\theta_t, \theta_h, \theta_w$	Rotation angles computed from each position coordinate. Different frequencies for different dimensions of the embedding.
$\odot$	Element-wise (Hadamard) multiplication — multiply corresponding elements.
$\hat{\mathbf{x}}$	The "rotated complement" — a rearranged version of $\mathbf{x}$ (pairs of dimensions are swapped with sign changes).
$\cos, \sin$	The trigonometric functions create a **rotation** in 2D subspaces of the embedding. Each pair of dimensions is rotated by an angle determined by position.

What Rotary Position Embedding Does

Think of a clock.
Standard position encoding adds a number ("you are token #5"). RoPE instead **rotates** the token vector by an angle proportional to its position. When two tokens compute attention ($\mathbf{Q}\cdot\mathbf{K}$), the dot product naturally captures the **distance** between them (because rotation differences encode positional differences).
MRoPE extends this to **3 axes**: time, height, width. The embedding dimension is split into 3 groups, and each group is rotated based on one of the three coordinates. This lets the model understand both spatial and temporal position simultaneously.
Key insight: for pure text (no visual input), all three coordinates are set equal ($t = h = w$), which reduces MRoPE to standard 1D RoPE.

Equation 12 — Visual-Language Fusion (Eq. 3.12)

The Equation

$$\mathbf{v}_{\text{lang}} = \text{MLP}_{\text{merge}}(\mathbf{v}_{\text{enc}}), \quad \mathbf{v}_{\text{lang}} \in \mathbb{R}^{N_v' \times D_{\text{LM}}}$$

What Every Symbol Means

$\mathbf{v}_{\text{enc}}$	Visual tokens from the Vision Encoder (output of all the window attention layers).
$\text{MLP}_{\text{merge}}$	A Multi-Layer Perceptron (small neural network with 2–3 linear layers) that projects the vision tokens into the language model's space.
$D_{\text{LM}} = 3584$	The hidden dimension of the Qwen2.5 language model decoder. This is the "language space."
N'_v	Number of visual tokens after merging (may differ from N_v if tokens are pooled/compressed).
$\mathbf{v}_{\text{lang}}$	The visual tokens now live in the same vector space as text tokens — they can be mixed together.

This is the bridge between vision and language. After this projection, visual tokens and text tokens are interleaved into a single sequence and processed jointly by the language model decoder.

Equation 13 — Language Model Training Objective (Eq. 3.13)

The Equation

$$\mathcal{L}_{\text{LM}} = -\sum_{t=1}^{T_{\text{seq}}} \log p_\theta(x_t \mid x_{<t}, \mathbf{v}_{\text{lang}})$$

What Every Symbol Means

T_{seq}	Total length of the text sequence.
x_t	The t-th text token in the sequence.
$x_{<t}$	All text tokens before position t (the "history").
$\mathbf{v}_{\text{lang}}$	The visual tokens (providing visual context).
$p_\theta(x_t \mid x_{<t}, \mathbf{v}_{\text{lang}})$	The model's predicted probability of token x_t given all previous tokens and the visual input.
log	Natural logarithm. Taking the log converts multiplication of probabilities into sums (numerically stable).
$-$	Negative sign makes it a loss to **minimize**. Minimizing the negative log-probability is equivalent to maximizing the probability of the correct next token.
$\sum$	Sum over all positions — the model must predict every token correctly.

In plain English: The model is trained to predict the next word given the image

and all previous words. This is standard autoregressive language model training.

Equation 14 — NF4 Quantization (Eq. 3.14)

The Equation

$$\mathbf{W}_{\text{NF4}} = \text{quantize}_{\text{NF4}}\left(\frac{\mathbf{W}}{s}\right) \cdot s, \quad s = \frac{\max(|\mathbf{W}_{\text{block}}|)}{q_{\max}}$$

What Every Symbol Means

$\mathbf{W}$	Original full-precision (float16) model weights.
$\mathbf{W}_{\text{block}}$	A small block of weights (e.g., 64 values). Quantization is done block-by-block, not globally.
s	Per-block **scaling factor**. It's the ratio of the maximum absolute value in the block to the maximum representable value in 4-bit.
$\mathbf{W}/s$	Normalized weights — divided by s to fit into the [-1, 1] range.
$\text{quantize}_{\text{NF4}}(\cdot)$	Maps each normalized value to the nearest of the 16 NF4 quantization levels ($2^4 = 16$ levels for 4-bit). These 16 levels are specifically chosen to minimize error for normally distributed values.
$\cdot\, s$	Multiply back by s to restore the original scale.
$q_{\max}$	Maximum quantile value in the NF4 data type.

Why This Matters: Memory Savings

Without quantization: 7B parameters × 2 bytes (float16) = **14 GB**
With NF4 (4-bit): 7B parameters × 0.5 bytes = **3.5 GB** + scaling factors ≈ **4 GB total**
Your GPU has 15.6 GB. Without quantization, the model alone fills 14 GB, leaving almost no room for input data and intermediate computations. With NF4, the model takes only 4 GB, leaving 11.6 GB for actual work.

Equation 15 — Qwen Feature Extraction (Eq. 3.15)

The Equation

$$\mathbf{f}_{\text{Qwen}} = \frac{1}{T_{\text{seq}}} \sum_{t=1}^{T_{\text{seq}}} \mathbf{h}_t^{(L)}, \quad \mathbf{f}_{\text{Qwen}} \in \mathbb{R}^{3584}$$

What Every Symbol Means

$\mathbf{h}_t^{(L)}$	The hidden state vector at position t from the **final** decoder layer L. Each is a 3584-dimensional vector.
T_{seq}	Total sequence length (visual tokens + text tokens combined).
$\sum_{t=1}^{T_{\text{seq}}}$	Sum over all positions in the sequence.
$\frac{1}{T_{\text{seq}}}$	Divide by the count to get the **average** (mean pooling).
$\mathbf{f}_{\text{Qwen}}$	The final feature vector for one video clip: a single vector of 3584 numbers.

In plain English: Take the last layer's output at every position, average them all together into one vector. This single 3584-d vector is the "summary" of the entire video clip.

Feature Preprocessing: Frozen Protocol and PCA

Equation 16 — Frozen Feature Protocol

The Equations

$$\mathbf{z}_i = f(\mathbf{x}_i; \boldsymbol{\Theta}_{\text{frozen}}), \quad \nabla_{\boldsymbol{\Theta}} \mathcal{L} = \mathbf{0}$$

$$\mathbf{W}^*_{\text{head}} = \arg\min_{\mathbf{W}_{\text{head}}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(g(\mathbf{z}_i; \mathbf{W}_{\text{head}}), y_i\right)$$

What Every Symbol Means

First equation:

$\mathbf{x}_i$	The i-th input video clip.
$f(\cdot\,; \boldsymbol{\Theta}_{\text{frozen}})$	The backbone model (V-JEPA 2 or Qwen2.5-VL) with frozen parameters. The semicolon means "parameterized by."
$\mathbf{z}_i$	The extracted feature vector for clip i.
$\nabla_{\boldsymbol{\Theta}} \mathcal{L} = \mathbf{0}$	"The gradient of the loss with respect to $\boldsymbol{\Theta}$ is zero." This formally means: **we do not update the backbone weights**. No gradients flow through the backbone.

Second equation:

$g(\cdot\,; \mathbf{W}_{\text{head}})$	The downstream classifier (SVM, MLP, HAF-STD Net, etc.) parameterized by $\mathbf{W}_{\text{head}}$.
y_i	True label for clip i.
$\mathcal{L}(\cdot, \cdot)$	Loss function (cross-entropy) comparing prediction with true label.
$\frac{1}{N} \sum_{i=1}^{N}$	Average loss over all N training samples.
$\arg\min$	"Find the $\mathbf{W}_{\text{head}}$ that minimizes this."
$\mathbf{W}^*_{\text{head}}$	The optimal classifier weights after training.

Why Freeze the Backbone?

Think of the backbone (V-JEPA 2 / Qwen) as a **camera lens**. It was designed and calibrated at a factory (pretrained on millions of videos). You don't grind new glass for each photo shoot — you just choose the right film (classifier) for the job. Freezing has practical benefits: (1) saves GPU memory (no gradient storage), (2) prevents overfitting on small datasets, (3) enables fair comparison between different classifiers using the same fixed features.

Equation 17 — Standardization (Eq. 3.16)

The Equation

$$\tilde{\mathbf{z}}_{ij} = \frac{z_{ij} - \mu_j}{\sigma_j}, \quad \mu_j = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} z_{ij}, \quad \sigma_j = \sqrt{\frac{1}{N_{\text{train}} - 1} \sum_{i=1}^{N_{\text{train}}} (z_{ij} - \mu_j)^2}$$

What Every Symbol Means

z_{ij}	Raw feature value for sample i, dimension j.
μ_j	Mean of dimension j computed over the training set. Add up all training samples' values for that dimension, divide by count.
σ_j	Standard deviation of dimension j. Measures how "spread out" the values are.
$\tilde{\mathbf{z}}_{ij}$	Standardized value. After transformation, dimension j has mean $= 0$ and standard deviation $= 1$ across the training set.
N_{train}	Number of training samples (not test samples — to avoid data leakage).
$N_{\text{train}} - 1$	Bessel's correction — using $N - 1$ instead of N gives an unbiased estimate of the population variance.

Numerical Example

Suppose dimension $j = 42$ has these values across 5 training samples: $[10, 20, 30, 40, 50]$.

$\mu_{42} = \frac{10+20+30+40+50}{5} = 30$

$\sigma_{42} = \sqrt{\frac{(10-30)^2+(20-30)^2+\ldots+(50-30)^2}{5-1}} = \sqrt{\frac{400+100+0+100+400}{4}} = \sqrt{250} = 15.81$

The value $z_{1,42} = 10$ becomes: $\tilde{z}_{1,42} = \frac{10-30}{15.81} = -1.265$

Now all dimensions have the same scale, so dimension 42 (which ranged 10–50) and dimension 99 (which might range 0.001–0.005) are treated equally by PCA and the classifiers.

Equations 18–20 — PCA (Eqs. 3.17–3.19)

Covariance Matrix (Eq. 3.17)

$$\mathbf{C} = \frac{1}{N_{\text{train}} - 1} \tilde{\mathbf{Z}}_{\text{train}}^{\top} \tilde{\mathbf{Z}}_{\text{train}} \in \mathbb{R}^{D \times D}$$

What This Does

$\tilde{\mathbf{Z}}_{\text{train}}$	The standardized feature matrix. Shape: $N_{\text{train}} \times D$ (samples $\times$ dimensions).
$\tilde{\mathbf{Z}}_{\text{train}}^{\top}$	Its transpose. Shape: $D \times N_{\text{train}}$.
$\tilde{\mathbf{Z}}^{\top}\tilde{\mathbf{Z}}$	Matrix multiplication. The result is a $D \times D$ matrix where entry (i, j) measures how correlated dimensions i and j are.
$\mathbf{C}$	The covariance matrix. Diagonal entries = variance of each dimension. Off-diagonal entries = covariance (correlation) between pairs of dimensions.

For V-JEPA 2 ($D = 1280$): $\mathbf{C}$ is a 1280×1280 matrix with 1,638,400 entries.

Variance Retention Threshold (Eq. 3.18)

$$k^* = \arg\min_k \left(\frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{D} \lambda_j} \geq 0.95 \right)$$

What This Does

λ_j	The j-th eigenvalue of $\mathbf{C}$, sorted in descending order. Each eigenvalue represents how much "information" (variance) the j-th principal component captures.
$\sum_{j=1}^{k} \lambda_j$	Total variance captured by the first k components.
$\sum_{j=1}^{D} \lambda_j$	Total variance across all D components.
The ratio	The fraction of total variance captured by keeping only k components.
≥ 0.95	We want at least 95% of the information preserved.
k^*	The smallest k that achieves this. This is how many dimensions we keep.

Your Actual Results

Configuration	Original D	Reduced k^*	Compression Ratio
V-JEPA 2 on FineGym	1,280	90	14.2×
Qwen2.5-VL on FineGym	3,584	142	25.2×
V-JEPA 2 on HMDB51	1,280	286	4.5×
Qwen2.5-VL on HMDB51	3,584	719	5.0×

HMDB51 needs more components because 51 classes require more information to distinguish than 4 classes.

PCA Projection (Eq. 3.19)

$$\mathbf{Z}_{\text{PCA}} = \tilde{\mathbf{Z}}\,\mathbf{V}_{:k^*} \in \mathbb{R}^{N \times k^*}$$

What This Does

$\tilde{\mathbf{Z}}$	Standardized feature matrix. Shape: $N \times D$.
$\mathbf{V}_{:k^*}$	The first k^* eigenvectors (principal components) of $\mathbf{C}$, stacked as columns. Shape: $D \times k^*$.
Matrix multiplication	$[N \times D] \times [D \times k^*] = [N \times k^*]$
$\mathbf{Z}_{\text{PCA}}$	The reduced features. Same number of samples (N), but each sample now has only k^* dimensions instead of D.

This is a simple matrix multiplication that projects each 1280-d (or 3584-d) feature vector onto the k^* most informative directions found by PCA.

HAF-STD Net: The Novel Classifier

Equation 21 — Haar Wavelet Transform

The Equation

$$\mathbf{a}_k = \frac{1}{2}(\mathbf{h}_{2t} + \mathbf{h}_{2t+1}), \qquad \mathbf{d}_k = \frac{1}{2}(\mathbf{h}_{2t} - \mathbf{h}_{2t+1})$$

What Every Symbol Means

$\mathbf{h}_{2t}, \mathbf{h}_{2t+1}$	Two consecutive temporal frames' feature vectors at encoder layer k.
$\mathbf{a}_k$	**Approximation coefficient** (low-frequency). The *average* of two consecutive frames. Captures what *stays the same* — static appearance, background, body posture.
$\mathbf{d}_k$	**Detail coefficient** (high-frequency). The *difference* between two consecutive frames. Captures what *changes* — motion, dynamics, movement direction.
k	Encoder layer index.
$\frac{1}{2}$	Normalization factor.

Numerical Example

Suppose at layer k, frame 1 has feature value $\mathbf{h}_0 = [0.8, 0.2]$ and frame 2 has $\mathbf{h}_1 = [0.6, 0.9]$:
$\mathbf{a}_k = \frac{1}{2}([0.8, 0.2] + [0.6, 0.9]) = \frac{1}{2}[1.4, 1.1] = [0.7, 0.55]$ (average: what's stable)
$\mathbf{d}_k = \frac{1}{2}([0.8, 0.2] - [0.6, 0.9]) = \frac{1}{2}[0.2, -0.7] = [0.1, -0.35]$ (difference: what changed)
The approximation $\mathbf{a}_k$ shows both features are moderate. The detail $\mathbf{d}_k$ shows the first feature barely changed ($+0.1$) while the second changed dramatically (-0.35).

Why This Decomposition?

Different actions have different appearance vs. motion signatures:

- **Vault:** Low appearance change (same runway and landing mat) but *explosive* motion signal (sprint → flip → land).
- **Balance Beam:** Relatively static appearance (narrow beam) with *subtle* motion (small steps, turns).

By separating appearance ($\mathbf{a}$) from motion ($\mathbf{d}$), the model can learn which signal matters more for each class.

Equation 22 — Depth Representation Velocity

The Equation

$$\Delta \mathbf{H}_k = \bar{\mathbf{H}}_{k+1} - \bar{\mathbf{H}}_k$$

What Every Symbol Means

$\bar{\mathbf{H}}_k$	Temporally mean-pooled hidden state at encoder layer k. A single vector summarizing the entire temporal sequence at depth k.
$\bar{\mathbf{H}}_{k+1}$	Same thing at the *next deeper* layer.
$\Delta \mathbf{H}_k$	The difference between consecutive layers. This is the "velocity" of how the representation changes as it passes deeper into the network.
k	Ranges over the 4 selected layers: $k \in \{L/4, L/2, 3L/4\}$ (3 velocity vectors from 4 layers).

Why "Velocity"?

As information flows from layer 8 $\rightarrow$ 16 $\rightarrow$ 24 $\rightarrow$ 32, the representation changes. For a vault video, the early layers might encode "person running" while later layers encode "gymnast performing vault." The **rate of change** between layers is different for different actions.

Think of $\Delta \mathbf{H}_k$ like the "derivative" of representation with respect to depth. It captures how quickly and in which direction the model's "understanding" evolves — and this evolution pattern is action-class-dependent.

Equation 23 — Adaptive Routing Fusion

The Equation

$$\mathbf{w} = \text{softmax}\left(\text{MLP}\left([\mathbf{s}_1; \mathbf{s}_2; \ldots; \mathbf{s}_{12}]\right)\right), \qquad \mathbf{f} = \sum_{i=1}^{12} w_i \cdot \mathbf{s}_i$$

What Every Symbol Means

$\mathbf{s}_1, \ldots, \mathbf{s}_{12}$	The 12 stream vectors: 4 layers $\times$ 3 streams per layer (appearance $\mathbf{a}_k$, motion $\mathbf{d}_k$, velocity $\Delta\mathbf{H}_k$). Each has been projected through its own linear head.
$[\cdot;\cdot;\ldots;\cdot]$	Concatenation of all 12 vectors into one long vector.
$\text{MLP}(\cdot)$	A small neural network that takes the concatenated vector and outputs 12 scalar scores.
$\text{softmax}(\cdot)$	Converts the 12 scores into probabilities that sum to 1. These are the **attention weights** — how much to trust each stream.
$\mathbf{w} = [w_1, \ldots, w_{12}]$	The learned fusion weights. w_i is the importance of stream i.
$\sum_{i=1}^{12} w_i \cdot \mathbf{s}_i$	Weighted sum of all 12 streams. Streams with higher weights contribute more.
$\mathbf{f}$	The final fused feature vector for classification.

Why Adaptive (Input-Dependent) Routing?

Different inputs need different stream combinations. For a vault video, motion streams ($\mathbf{d}_k$) from later layers might be most important. For distinguishing balance beam from uneven bars, appearance streams ($\mathbf{a}_k$) from early layers might be key. The MLP learns to **look at the input** and decide: "For this particular video, which combination of appearance, motion, and velocity information is most useful?" The weights change for every input — this is what makes it "adaptive."

Equation 24 — Cosine Similarity Classification Head

The Equation

$$p(y = c \mid \mathbf{f}) = \frac{\exp\left(\tau \cdot \hat{\mathbf{f}}^T \hat{\mathbf{w}}_c\right)}{\sum_{j=1}^{C} \exp\left(\tau \cdot \hat{\mathbf{f}}^T \hat{\mathbf{w}}_j\right)}$$

What Every Symbol Means

$\mathbf{f}$	The fused feature vector from the adaptive routing.
$\hat{\mathbf{f}}$	The L_2-normalized version: $\hat{\mathbf{f}} = \mathbf{f}/\|\mathbf{f}\|$. After normalization, the vector has unit length (lies on a hypersphere).
$\hat{\mathbf{w}}_c$	The L_2-normalized learnable prototype vector for class c. Each class has its own prototype.
$\hat{\mathbf{f}}^T\hat{\mathbf{w}}_c$	Dot product of two unit vectors = **cosine similarity**. Ranges from -1 (opposite) to $+1$ (identical). Measures how "similar" the input is to class c's prototype.
τ	Learnable **temperature** parameter. Scales the similarities to control how "sharp" the probability distribution is. Higher τ = more confident predictions.
$\exp(\cdot)$	Exponential function. Converts similarities into positive values.
$\sum_{j=1}^{C} \exp(\cdot)$	Sum over all C classes in the denominator. This is the softmax normalization that ensures probabilities sum to 1.
$p(y = c\|\mathbf{f})$	Final predicted probability that the input belongs to class c.

Why Cosine Similarity Instead of a Linear Layer?

A standard linear classifier computes $\mathbf{W}^T\mathbf{f} + \mathbf{b}$, which mixes magnitude and direction. A feature vector that happens to have large magnitude gets different treatment than one with small magnitude.

Cosine similarity **only cares about direction**, not magnitude. By normalizing both $\mathbf{f}$ and $\mathbf{w}_c$, we place all features and class prototypes on a unit hypersphere. Classification becomes: "which class prototype is the input vector pointing most closely toward?"

This is more robust when feature magnitudes vary (which they do across different video clips).

Training Techniques

Equation 25 — Label Smoothing

The Equation

$$y_c^{\text{smooth}} = (1-\epsilon)y_c + \frac{\epsilon}{C}$$

What Every Symbol Means

y_c	The original one-hot label for class c. It's 1 for the correct class, 0 for all others.
$\epsilon = 0.1$	Smoothing parameter. Controls how much probability to "spread" to other classes.
C	Number of classes (4 for FineGym, 51 for HMDB51).
$(1-\epsilon)$	The correct class keeps $1 - 0.1 = 0.9$ of the probability.
$\frac{\epsilon}{C}$	Each class (including the correct one) gets an extra $\frac{0.1}{C}$.
y_c^{smooth}	The smoothed target probability for class c.

Numerical Example: FineGym (C=4)

True label: Balance Beam (class 0).
Without smoothing (one-hot): $[1.0, 0.0, 0.0, 0.0]$
With smoothing ($\epsilon = 0.1$):

- Correct class: $(1-0.1) \times 1 + \frac{0.1}{4} = 0.9 + 0.025 = 0.925$
- Other classes: $(1-0.1) \times 0 + \frac{0.1}{4} = 0 + 0.025 = 0.025$

Smoothed target: $[0.925, 0.025, 0.025, 0.025]$
Sum: $0.925 + 3 \times 0.025 = 1.0$ ✓
The model is now trained to be 92.5% confident (not 100%), which prevents over-confident predictions and improves generalization.

Equation 26 — Feature-Space Mixup

The Equation

$$\tilde{\mathbf{z}} = \lambda \mathbf{z}_i + (1-\lambda)\mathbf{z}_j, \quad \lambda \sim \text{Beta}(0.2, 0.2)$$

What Every Symbol Means

$\mathbf{z}_i, \mathbf{z}_j$	Two different training samples' feature vectors.
λ	A random mixing coefficient drawn from the Beta(0.2, 0.2) distribution. Most values will be near 0 or 1 (i.e., mostly one sample or the other, with occasional 50/50 mixes).
$\tilde{\mathbf{z}}$	The mixed feature vector — a blend of two samples.
$\text{Beta}(0.2, 0.2)$	A U-shaped distribution. Unlike Beta(1,1) which is uniform, Beta(0.2,0.2) concentrates probability near 0 and 1, with some probability in between.

The loss is also mixed: $\mathcal{L} = \lambda \cdot \mathcal{L}(\tilde{\mathbf{z}}, y_i) + (1 - \lambda) \cdot \mathcal{L}(\tilde{\mathbf{z}}, y_j)$

Why: This creates "virtual" training examples between real ones, filling gaps in the feature space. It acts as a regularizer, reducing overfitting without collecting more data.

Equation 27 — 4D Mixup for HAF-STD Net

The Equation

$$\tilde{\mathbf{F}} = \lambda \cdot \mathbf{F}_i + (1 - \lambda) \cdot \mathbf{F}_{\pi(i)}, \quad \lambda \sim \text{Beta}(0.2, 0.2)$$

What This Adds

$\mathbf{F}_i$	Full 4D multi-layer tensor for sample i. Shape: $4 \times T \times 1280$.
$\pi(i)$	A random permutation index — $\mathbf{F}_{\pi(i)}$ is a randomly chosen different sample.
$\tilde{\mathbf{F}}$	The mixed 4D tensor. Every element is interpolated.

This is the same idea as standard mixup (Eq. 26) but applied to the full 4D tensor instead of a 1D feature vector. The interpolation happens *element-wise across all dimensions simultaneously* — preserving the multi-layer temporal structure while blending two samples.

Complete Equation Index

All 27 Equations at a Glance

#	Equation Name	Component	What It Does
1	Spatiotemporal Tokenization	V-JEPA 2	Video → token sequence
2	EMA Update	V-JEPA 2	Slowly updates target encoder
3	Transformer Block	V-JEPA 2	Attention + FFN per layer
4	V-JEPA 2 Objective	V-JEPA 2	Predict masked patches
5	Dense Per-Token Loss	V-JEPA 2.1	Per-token prediction supervision
6	Context Loss	V-JEPA 2.1	Regularize visible tokens
7	Total V-JEPA 2.1 Loss	V-JEPA 2.1	Combine predict + context
8	Multi-Layer Features	V-JEPA 2	Extract 4 depth layers
9	Patch Embedding	Qwen2.5-VL	Video → visual tokens
10	Window Attention	Qwen2.5-VL	Efficient local attention
11	MRoPE	Qwen2.5-VL	3D position encoding
12	Visual-Language Fusion	Qwen2.5-VL	Vision tokens → LM space
13	LM Training Objective	Qwen2.5-VL	Next-token prediction
14	NF4 Quantization	Qwen2.5-VL	16-bit → 4-bit compression
15	Feature Extraction	Qwen2.5-VL	Mean pool → 3584-d vector
16	Frozen Feature Protocol	Pipeline	Freeze backbone weights
17	Standardization	Pipeline	Zero-mean / unit-variance
18	Covariance Matrix	PCA	Compute feature correlations
19	Variance Retention	PCA	Find k^* components for 95%
20	PCA Projection	PCA	D-dim → k^*-dim
21	Haar DWT	HAF-STD Net	Appearance + motion streams
22	Depth Velocity	HAF-STD Net	Inter-layer deltas
23	Adaptive Routing	HAF-STD Net	Weighted fusion of 12 streams
24	Cosine Classification	HAF-STD Net	Normalized similarity head
25	Label Smoothing	Training	Soften one-hot targets
26	Feature Mixup	Training	Interpolate feature pairs
27	4D Mixup	Training	Extend mixup to 4D tensors